Week 10 — Final Project Part I

Code the Dream · Python Intro

Session Overview

Explore — ~30 min

- Final project overview and two-week arc
- Choosing an API
- Phase 1: fetch + parse
- Phase 2: CLI tool
- Code organization for a larger program

Apply — ~35 min

- Code-along: explore an API in the browser
- Code-along: fetch + parse function
- Code-along: basic CLI structure

The Two-Week Arc

The final project spans two weeks and builds on everything you've learned.

Week	What you build
10	Core program: fetch live API data + CLI tool
11	Extension: data visualization (A) or CSV export (B)

Week 11 builds directly on top of Week 10 — getting the core right this week matters.

Choosing an API

Use a public API that doesn't require an API key for the core features.

Good choices from the lessons:

- REST Countries (`restcountries.com`) — countries, capitals, populations
- Open-Meteo (`open-meteo.com`) — weather data by location
- Open Library (`openlibrary.org/api`) — books by author or title
- PokéAPI (`pokeapi.co`) — Pokémon stats and data
- NASA APOD (`api.nasa.gov`) — astronomy photos (free key)

Phase 0: Explore Before You Code

Before writing a single line of Python:

1. Open the API URL in your browser
2. Find an endpoint that returns interesting data
3. Identify the fields you'll use
4. Sketch your function signatures on paper

```
fetch_data()         → list of dicts
parse_response(raw) → list of clean dicts
search(data, term)  → list of matching dicts
```

Phase 1: Core Program

Fetch data and parse it into a list of clean dicts.

```
def fetch_countries():
    r = requests.get(URL, params=PARAMS, timeout=10)
    r.raise_for_status()
    return [
        {"name": c["name"]["common"],
         "region": c["region"],
         "population": c["population"]}
        for c in r.json()
    ]
```

Check for Understanding #1

What is the main job of the data-fetching function in Phase 1?

A) Print the raw API response to the terminal
B) Fetch data and return a clean list of dicts the CLI can work with
C) Build the menu and handle user input
D) Write the data to a CSV file

Phase 2: CLI Tool

Accept user input, apply an interaction, display results.

```
def main():
    print("Loading data...")
    data = fetch_countries()

    while True:
        print("\n1. Search by name  2. Filter by region  3. Quit")
        choice = input("Choose (1-3): ")
        if choice == "1":
            term = input("Search: ")
            results = search(data, term)
            display(results)
        elif choice == "3":
            break
```

Code Organization

Keep the four layers separate.

Layer	What goes here
Fetch	API call, raise_for_status, return raw data
Parse	Transform raw data into clean list of dicts
Logic	search, filter, sort — pure functions, no I/O
Display	print statements, formatting
main()	the while loop that ties it all together

Check for Understanding #2

Your `search(data, term)` function should return matching results. Where does the `print` go?

A) Inside `search()` — it loops through and prints matches
B) In `main()` — `search()` returns the matches, main() prints them
C) In `fetch_data()` — that's where the data lives
D) It doesn't matter where the print goes

Check for Understanding #3

What is the minimum commit history expected for this project?

A) One commit with everything finished
B) One commit per file
C) Separate commits for: getting the API working, parsing the data, and adding the CLI
D) Commits only when the whole project works

Time to Apply

Let's build the core program together.

Mentor 2 takes over

Assignment 10 Overview

Work in the starter repo (`python-intro-final-project`), not the homework repo.

Phase 1 — Core program

- Connect to a public API, parse response into a list of dicts
- Wrap in functions with clear parameters and return values
- Handle connection errors and bad status codes

Phase 2 — CLI tool

- At least one meaningful user interaction (filter, search, or compare)
- Handle unexpected input without crashing
- Required files: `main.py`, `requirements.txt`, `README.md`

Code-Along 1: Explore the API First

Open the endpoint in your browser before writing any Python.

```
https://restcountries.com/v3.1/all?fields=name,capital,region,population
```

- What does one item in the list look like?
- What's the data type of `capital`? (It's a list — watch out!)
- Which fields have missing values?
- What will you name your dict keys in the clean version?

Code-Along 2: Fetch + Parse Function

Write the data layer first — CLI comes after.

```
import requests

URL = "https://restcountries.com/v3.1/all"
PARAMS = {"fields": "name,capital,region,population"}

def fetch_countries():
    try:
        r = requests.get(URL, params=PARAMS, timeout=10)
        r.raise_for_status()
        return [{"name": c["name"]["common"],
                 "capital": (c.get("capital") or ["N/A"])[0],
                 "region": c.get("region", "Unknown"),
                 "population": c["population"]}
                for c in r.json()]
    except requests.exceptions.RequestException as e:
        print(f"Error: {e}")
        return []
```

Code-Along 3: Basic CLI Structure

Build the menu loop on top of the data.

```
def main():
    print("Loading data...")
    data = fetch_countries()
    if not data:
        print("No data loaded. Exiting.")
        return

    while True:
        print("\n=== Country Explorer ===")
        print("1. Search by name  2. Filter by region  3. Quit")
        choice = input("Choose (1–3): ").strip()
        if choice == "3":
            print("Goodbye!")
            break
        else:
            print("(Search and filter coming soon)")

main()
```

Extension Challenge

Ungraded extension: Add a third menu option — "Compare two countries" — that asks for two country names and prints a side-by-side comparison of their population and region.

Think about: how would you find an exact match by name in your list of dicts? A loop with `if c["name"].lower() == term.lower()`.

Wrap-Up

Today you learned:

- The two-phase project structure: core program → CLI
- Exploring an API before writing code
- Keeping fetch, parse, logic, and display in separate functions
- Starting the `main()` scaffold and building up incrementally

This week: Complete Phase 1 and Phase 2. Commit separately for each phase. Open a PR with a video demo link.

Next week: Extend your project with data visualization or a CSV export pipeline.